uc3m | Universidad Carlos III de Madrid

Master Degree in
Applied and Computational Mathematics
2025-2026

Master Thesis

Computational algorithms for pricing financial derivatives under local volatility models

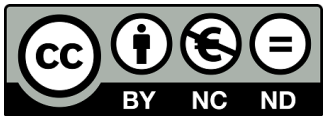
---

Jose Enrique Zafra Mena

Jorge Morón Vidal
Francisco Manuel Bernal Martínez

Madrid, 2026

AVOID PLAGIARISM
The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



SUMMARY

Keywords:

DEDICATION

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

1.1. Motivation

Although the thesis title emphasizes derivative pricing, the practical objective of this work is broader: to build a fast and coherent workflow for market calibration, valuation, and Greek computation under stochastic volatility. In the Heston setting, each of these tasks is numerically demanding on its own, and in practice they must be executed jointly and repeatedly.

This creates a central practical tension: high-fidelity numerical references are reliable, yet expensive when used at scale. The cost becomes especially visible in repetitive workloads such as calibration iterations across market surfaces, scenario analysis, and dense batches of sensitivities. As a result, computational latency can become a bottleneck even before model quality is exhausted.

For this reason, surrogate approaches are explored as acceleration layers on top of trusted numerical benchmarks. The goal is not to replace model-consistent finance with a black-box shortcut, but to preserve reference behavior within controlled error levels while reducing runtime enough for practical use.

Within this strategy, two priorities are central. First, robust market calibration, since all downstream valuation depends on the quality and stability of fitted parameters. Second, fast Greek estimation through PINN-based differentiable surrogates, so that sensitivities can be obtained efficiently without relying on repeated finite-difference repricing.

This thesis is developed as a research-oriented prototype rather than as a production deployment. The methodological pipeline (data generation, reference solvers, surrogate training, calibration, and benchmarking logic) is designed to be reusable beyond a single contract class. However, some components are intentionally case-specific: in particular, the Greeks block based on the PINN formulation is developed for European vanilla contracts under the Heston PDE constraints used in this work. This distinction clarifies from the start what is expected to generalize and what remains tied to the chosen pricing setting.

1.2. Main Objectives

This thesis focuses on the Heston stochastic-volatility framework and aims to evaluate computational methods under a consistent speed-accuracy trade-off perspective. The first objective is to build a reproducible end-to-end pipeline covering data preparation, reference numerical pricing, calibration, surrogate/PINN training, and benchmark execution. This objective is methodological: the full workflow must be easy to rerun and auditable

at each stage. add final references to scripts and modules once the pipeline layout is done. if needed, consolidate these references in an appendix table

The second objective is comparative. The thesis develops and evaluates surrogate approaches, including supervised neural surrogates and PINN-based formulations, against numerical references under common experimental conditions. The purpose is to determine where learned approximators provide practical acceleration without losing control of model-consistent behavior.

The third objective is empirical evaluation through quantitative metrics. Performance is assessed using pricing error, Greek error, computational cost, and numerical stability across the parameter regions and contract settings covered by the experiments. The goal is to characterize not only average accuracy, but also robustness and failure modes.

The fourth objective is to establish realistic operational boundaries. The thesis explicitly separates components that are sufficiently mature for applied use from those that remain exploratory, especially in relation to portability, maintenance cost, and reliability under distribution shifts.

Placeholder: High-level pipeline diagram

Market data and synthetic data preparation → Heston reference solvers (COS/PDE + IV inversion) → calibration and learned surrogates (ANN/PINN) → pricing and Greeks benchmarks.

Fig. 1.1. High-level pipeline of the thesis (placeholder).

Latency is treated as an informative benchmark dimension rather than as a production service-level target. Reported timings are interpreted in the context of the current research implementation, without hardware-specific optimization as a primary objective.

1.3. Key Contributions

This thesis makes four main contributions. First, it delivers a reproducible computational framework for Heston-based market calibration, reference pricing, surrogate training, and benchmarking under a unified experimental protocol. This framework is designed to make method comparisons auditable and repeatable.

Second, it provides an empirical comparison between numerical references and neural surrogates, including supervised ANN models and PINN-based formulations, under shared evaluation conditions. The comparison is used to characterize practical speed-accuracy trade-offs and to identify parameter regions where each approach remains reliable.

Third, it develops a PINN-based workflow for fast Greek computation in the vanilla European Heston setting, using automatic differentiation on a differentiable pricing sur-

rogate and validating the resulting sensitivities against reference numerical estimates.

Fourth, it documents the operational limits of the proposed methodology through an explicit robustness and validity analysis, including data-domain coverage, discretization effects, extrapolation risk, and dependence on benchmark design. This yields a clear boundary between components that are mature enough for applied use and components that remain exploratory.

1.4. Scope and Structure

The scope of this thesis is explicitly bounded to Heston-based pricing, calibration, and Greek estimation workflows under the data and contract settings defined in the experimental design. The focus is on methodological evaluation and reproducible benchmarking, not on full production deployment or broad asset-class coverage beyond the tested domain.

Validation is presented at a high level through the datasets used, the train/validation/test logic adopted in each learning stage, and the benchmark criteria applied to numerical and neural methods. The goal is to keep the comparison protocol transparent and consistent across chapters. finalize the benchmark list and the exact data period after the last data-quality review

The remainder of the thesis is organized as follows. Chapter 2 introduces the mathematical and financial background. Chapter 3 presents the supervised neural surrogate pricer under Heston. Chapter 4 covers the calibration architecture and workflow. Chapter 5 develops the PINN formulation for pricing and Greeks. Chapter 6 reports performance and benchmark results. Finally, Chapter 7 summarizes conclusions and future work.

2. MATHEMATICAL AND FINANCIAL BACKGROUND

2.1. Mathematical Framework For Option Pricing

This section defines the common mathematical setup used in the rest of the thesis for pricing European-style derivatives.

Time is continuous on $[0, T]$, with current time t and maturity T. Rather than introducing the full probabilistic machinery, we keep a pricing-focused notation that is sufficient for the methods used later in the thesis.

A global notation glossary is provided in Appendix C and will be extended as new symbols are introduced.

As a practical starting point for this thesis, we use the standard risk-neutral valuation formula for European claims with constant rate r [1, Ch. 8]:

$$V_t = e^{-r\tau}\,\mathbb{E}_t^{\mathbb{Q}}[\Psi]. \tag{2.1}$$

In the following sections, this framework is connected to three ingredients used throughout the thesis: the stochastic-calculus tools needed for the model equations, the Heston stochastic-volatility dynamics, and the characteristic-function/Fourier-COS pricing route.

2.2. Stochastic Calculus Background

This section introduces only the stochastic-calculus notions needed by the pricing methods used later. The presentation follows standard probability definitions and the transform-pricing notation adopted in [1, Ch. 1] and [2, Sec. 2].

For a real random variable X, the cumulative distribution function is

$$F_X(x) = \mathbb{P}(X \le x). \tag{2.2}$$

If F_X is differentiable, the associated probability density function is

$$f_X(x) = \frac{\partial F_X(x)}{\partial x}. \tag{2.3}$$

Following this notation, the characteristic function of X is

$$\phi_X(u) := \mathbb{E}\left[e^{iuX}\right] = \int_{-\infty}^{\infty} e^{iux} dF_X(x) = \int_{-\infty}^{\infty} e^{iux} f_X(x) dx, \tag{2.4}$$

for $u \in \mathbb{R}$.

Applying the logarithm to ϕ_X, we obtain the cumulant characteristic function

$$\zeta_X(u) = \log \mathbb{E}\left[e^{iux}\right] = \log \phi_X(u). \tag{2.5}$$

Its Maclaurin expansion is

$$\zeta_X(u) = \sum_{k=0}^{\infty} \frac{\partial^k \zeta_X(u)}{\partial u^k}\bigg|_{u=0} \frac{u^k}{k!} = \sum_{k=0}^{\infty} \zeta_k \frac{u^k}{k!}, \tag{2.6}$$

where $\zeta_k \equiv \frac{\partial^k \zeta_X(u)}{\partial u^k}\Big|_{u=0}$ is the k-th cumulant.

The moment generating function is

$$\mathcal{M}_X(u) := \phi_X(-iu) = \mathbb{E}\left[e^{ux}\right]. \tag{2.7}$$

Using $\mathcal{M}_X(u) = \phi_X(-iu)$ and $\zeta_X(u) = \log \mathcal{M}_X(u)$, cumulants can be computed from the characteristic function as

$$\zeta_k = \frac{\partial^k \zeta_X(u)}{\partial u^k}\bigg|_{u=0} = \frac{\partial^k}{\partial u^k} \log \phi_X(-iu)\bigg|_{u=0}. \tag{2.8}$$

These objects are central in the rest of this chapter: the characteristic function provides direct access to Fourier-domain pricing formulas, and low-order cumulants are later used to construct truncation intervals for the COS method.

2.3. The Heston Stochastic Volatility Model

In a general diffusion form, the Heston dynamics can be written as

$$dS_t = \mu S_t\, dt + \sqrt{v_t}\, S_t\, dW_t^{(S)}, \tag{2.9}$$

$$dv_t = \kappa(\bar{v} - v_t)\, dt + \gamma \sqrt{v_t}\, dW_t^{(v)}, \tag{2.10}$$

$$d\langle W^{(S)}, W^{(v)} \rangle_t = \rho\, dt. \tag{2.11}$$

The parameter μ denotes a generic drift under the original measure. For pricing, we move to the risk-neutral measure $\mathbb{Q}$, under which the discounted asset price is a martingale and, in the no-dividend case considered in this thesis, the drift becomes r:

$$dS_t = rS_t\, dt + \sqrt{v_t}\, S_t\, dW_t^{(S,\mathbb{Q})}. \tag{2.12}$$

This is the form used in the pricing formulas of the following sections [1], [3].

Here, v_t is the instantaneous variance (equivalent to the v_t notation used in later data-driven chapters). The variance process is of CIR type, so it is mean-reverting toward $\bar{v}$ at speed κ, with volatility of variance controlled by γ.

In this thesis, implied volatility is defined as the value σ_{imp} that reproduces a given option price when inserted into the Black–Scholes formula with the same contract and market inputs [4]:

$$V_{BS}(S_0, K, r, \tau, \sigma_{imp}) = V_{\text{target}}. \tag{2.13}$$

Therefore, implied volatility is a quoting transformation of prices (not a structural parameter of Heston). The numerical inversion used to compute σ_{imp} is detailed in Section 2.6.

Parameter interpretation is directly linked to the implied-volatility (IV) surface:

TABLE 2.1. INTERPRETATION OF HESTON PARAMETERS

Parameter	Interpretation	Main effect on IV surface
ρ	Instantaneous correlation between spot and variance shocks.	Controls skew: more negative ρ typically implies stronger left skew.
γ	Volatility of variance (*vol-of-vol*).	Controls curvature and short-maturity smile amplitude; larger γ usually increases convexity.
$\bar{\nu}$	Long-run mean level of the variance process.	Sets the global IV level around which the surface oscillates across maturities.
ν_0	Initial variance level at valuation time (ν_{t_0}).	Anchors short-maturity ATM IV and strongly affects the front end of the term structure.
κ	Speed of mean reversion of ν_t toward $\bar{\nu}$.	Controls how fast maturities converge to long-run variance, affecting term-structure smoothing.

Source: Based on [3] and [5, Chs. 2–3]

A common positivity condition for the CIR variance process is the Feller condition

$$2\kappa\bar{\nu} \geq \gamma^2. \tag{2.14}$$

In practice, calibrated parameter sets may violate this inequality while still producing usable market fits; therefore, it is treated as a diagnostic and stability indicator rather than as a hard calibration constraint in many numerical workflows [5, Chs. 2–3].

For pricing, the key advantage of Heston is that the log-price characteristic function is available in closed (semi-analytical) form [3]. This makes the model especially suitable for Fourier-domain valuation, which is the computational route adopted in the next sections.

2.4. Pricing Via Characteristic Functions

When the characteristic function is available in closed or semi-closed form, pricing in the Fourier domain becomes a natural alternative to direct density integration. Following [2,

Sec. 2], we write the option value in terms of the conditional transition density of the terminal state variable.

Using the notation adopted in the next section, with $x = X(t_0)$ and $y = X(T)$, the discounted valuation formula is

$$V(t_0, x) = e^{-r\tau} \int_{\mathbb{R}} V(T, y)\, f_X(T, y; t_0, x)\, dy. \tag{2.15}$$

The corresponding conditional characteristic function is

$$\phi_X(u; t_0, x) := \mathbb{E}^{\mathbb{Q}}_{t_0}\Big[e^{iuX(T)} \mid X(t_0) = x\Big] = \int_{\mathbb{R}} e^{iuy} f_X(T, y; t_0, x)\, dy. \tag{2.16}$$

Under standard integrability conditions, the density can be recovered by Fourier inversion:

$$f_X(T, y; t_0, x) = \frac{1}{2\pi} \int_{\mathbb{R}} e^{-iuy} \phi_X(u; t_0, x)\, du. \tag{2.17}$$

Let $\hat{V}_T(u) := \int_{\mathbb{R}} e^{-iuy} V(T, y)\, dy$ denote the Fourier transform of the payoff at maturity. Substituting the inverse representation of f_X into the pricing integral and exchanging the order of integration gives

$$V(t_0, x) = e^{-r\tau} \frac{1}{2\pi} \int_{\mathbb{R}} \phi_X(u; t_0, x)\, \hat{V}_T(u)\, du. \tag{2.18}$$

Compared with PDE solvers, this approach avoids full space-time gridding. Compared with Monte Carlo, it is deterministic and does not introduce sampling noise. In repeated-pricing tasks (surface generation, calibration loops, and large synthetic datasets), this is a major practical advantage.

The Fourier-COS method used in the next section is a specific spectral discretization of this transform-based pricing framework: it replaces the continuous integral by a finite cosine expansion over a truncated interval, preserving high accuracy with efficient vectorized evaluation.

2.5. The Fourier-COS method

The Fourier-COS method approximates transition densities and option values by a cosine series on a finite interval [2, Secs. 3–4]. Its practical appeal in this thesis is that it combines spectral convergence with efficient vectorized evaluation once the model characteristic function is available.

Let $x = X(t_0)$ and $y = X(T)$, and consider a truncation interval $[a, b]$ that concentrates most of the probability mass of $X(T) \mid X(t_0) = x$. On this interval, the conditional density is approximated by

$$\hat{f}_X(y) = \sum_{k=0}^{N-1}{}' \bar{F}_k \cos\left(\pi k \frac{y - a}{b - a}\right), \tag{2.19}$$

where $\sum'$ indicates that the $k = 0$ term is weighted by $1/2$, and the cosine coefficients are obtained from the characteristic function:

$$\bar{F}_k := \frac{2}{b-a}\Re\left[\phi_X\left(\frac{\pi k}{b-a}\right)\exp\left(-\frac{i\pi ak}{b-a}\right)\right]. \tag{2.20}$$

Starting from

$$V(t_0, x) = e^{-r\tau}\int_{\mathbb{R}} V(T, y) f_X(T, y; t_0, x)\, dy, \tag{2.21}$$

the COS discretization yields

$$V(t_0, x) \approx e^{-r\tau}\sum_{k=0}^{N-1}{}' \Re\left[\phi_X\left(\frac{\pi k}{b-a}\right)\exp\left(-\frac{i\pi ak}{b-a}\right)\right] H_k, \tag{2.22}$$

where the payoff coefficients are

$$H_k := \frac{2}{b-a}\int_a^b V(T, y)\cos\left(\frac{y-a}{b-a}\pi k\right) dy. \tag{2.23}$$

For plain-vanilla options, H_k has closed-form expressions, so the numerical effort is concentrated in evaluating the characteristic function on a fixed frequency grid [2, Sec. 3].

The choice of $[a, b]$ controls truncation error. Following the cumulant-based rule in [2, Sec. 5], we use

$$[a, b] = \left[c_1 - L\sqrt{c_2 + \sqrt{c_4}},\ c_1 + L\sqrt{c_2 + \sqrt{c_4}}\right], \tag{2.24}$$

with c_1, c_2, c_4 the first, second, and fourth cumulants of $X(T)$ and $L > 0$ a scaling parameter. This connects directly with the cumulant machinery introduced in Section 2.2. In the numerical experiments of this thesis, the integration interval is selected using the refined cumulant-based strategy detailed in Appendix A.

From a numerical viewpoint, it is useful to separate two error sources:

$$\left|V - V_N^{\mathrm{COS}}\right| \le \underbrace{\varepsilon_{\mathrm{trunc}}([a, b])}_{\text{domain truncation}} + \underbrace{\varepsilon_N}_{\text{finite cosine series}}. \tag{2.25}$$

Increasing L reduces $\varepsilon_{\mathrm{trunc}}$ but may require larger N to keep ε_N small; reducing L has the opposite effect. In practice, we choose (L, N) jointly and verify stability, especially for extreme strikes and short maturities, where numerical artifacts appear first.

Implementation is fully vectorized: frequencies $u_k = \pi k/(b-a)$, exponential phase factors, and payoff coefficients are precomputed once per maturity, then reused across strike grids. This is one of the reasons COS is well suited for repeated-pricing workflows such as calibration loops and synthetic dataset generation.

2.5.1. The COS Method Applied To The Heston Model

Following the notation of Sections 2.4–2.5 and [2, Sec. 3], define

$$u_k := \frac{k\pi}{b-a}, \qquad x := \log\!\left(\frac{S_0}{K}\right), \qquad y := \log\!\left(\frac{S_T}{K}\right).$$

Under Heston dynamics, the conditional characteristic function of y can be written in affine form [3], [5]

$$\varphi_H(u;\tau,x,v_0) = \exp(C(u,\tau) + D(u,\tau)v_0 + iux)\,, \tag{2.26}$$

with

$$d(u) = \sqrt{(\kappa - i\rho\gamma u)^2 + \gamma^2\,(u^2 + iu)}, \tag{2.27}$$

$$g(u) = \frac{\kappa - i\rho\gamma u - d(u)}{\kappa - i\rho\gamma u + d(u)}, \tag{2.28}$$

$$C(u,\tau) = iur\tau + \frac{\kappa\bar{v}}{\gamma^2}\left[(\kappa - i\rho\gamma u - d(u))\,\tau - 2\log\!\left(\frac{1 - g(u)e^{-d(u)\tau}}{1 - g(u)}\right)\right], \tag{2.29}$$

$$D(u,\tau) = \frac{\kappa - i\rho\gamma u - d(u)}{\gamma^2}\left(\frac{1 - e^{-d(u)\tau}}{1 - g(u)e^{-d(u)\tau}}\right). \tag{2.30}$$

This is the standard semi-closed representation used in Fourier pricing; numerically, consistency in branch selection for $d(u)$ and the complex logarithm is required for stability [5, Chs. 2–3].

The COS price for one strike can then be written as

$$V(t_0,x) \approx Ke^{-r\tau}\,{\sum_{k=0}^{N-1}}'\,\Re\Big[\varphi_H(u_k;\tau,x,v_0)\,e^{-iu_k a}\Big]\,U_k, \tag{2.31}$$

where U_k are payoff-dependent COS coefficients. For plain-vanilla options:

$$U_k^{\text{call}} = \frac{2}{b-a}\,(\chi_k(0,b) - \psi_k(0,b))\,, \qquad U_k^{\text{put}} = \frac{2}{b-a}\,(-\chi_k(a,0) + \psi_k(a,0))\,, \tag{2.32}$$

with

$$\chi_k(c,d) = \frac{1}{1+u_k^2}\Big[e^d(\cos(u_k(d-a)) + u_k\sin(u_k(d-a))) - e^c(\cos(u_k(c-a)) + u_k\sin(u_k(c-a)))\Big], \tag{2.33}$$

$$\psi_k(c,d) = \begin{cases} d - c, & k = 0, \\ \dfrac{b-a}{k\pi}\,[\sin(u_k(d-a)) - \sin(u_k(c-a))]\,, & k \geq 1. \end{cases} \tag{2.34}$$

In the implementation of this thesis, data generation is run on European puts as a numerical convention, mainly associated with the COS-pricing setup used in the project configuration.

This subsection establishes the analytical Heston-COS pricing block used as the reference framework in the following chapters.

2.5.2. Semi-Analytical Greeks from the Heston Characteristic Function

For validation purposes, Greeks can also be defined in the semi-analytical Heston framework, i.e., directly from the characteristic-function pricing representation. For a European call, a standard form is [3] and is developed in [5, Chapter 1]

$$C(S_0, K, \tau) = S_0 \Pi_1 - K e^{-r\tau} \Pi_2, \tag{2.35}$$

where

$$\Pi_j = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \Re\left(\frac{e^{-iu \log K} \Psi_j(u)}{iu} \right) du, \qquad j \in \{1, 2\}, \tag{2.36}$$

with

$$\Psi_2(u) = \phi_X(u), \qquad \Psi_1(u) = \frac{\phi_X(u - i)}{\phi_X(-i)}, \tag{2.37}$$

and ϕ_X the characteristic function of $X_T = \log S_T$ under Heston [3]; see also [5, Chapter 1].

Let p denote a generic model/input variable ($r, \tau, v_0, \rho, \kappa, \gamma, \bar{v}$, etc.). Under standard regularity and integrability conditions for affine models, differentiation under the integral sign yields [5, Chapter 11]

$$\partial_p \Pi_j = \frac{1}{\pi} \int_0^\infty \Re\left(\frac{e^{-iu \log K}}{iu} \partial_p \Psi_j(u) \right) du. \tag{2.38}$$

Therefore, Greeks are obtained by combining the derivatives of Π_1, Π_2 through the pricing identity [5, Chapter 11]. For example:

$$\Delta_{\text{call}} = \partial_{S_0} C = \Pi_1 + S_0 \, \partial_{S_0} \Pi_1 - K e^{-r\tau} \partial_{S_0} \Pi_2, \tag{2.39}$$

$$\Gamma_{\text{call}} = \partial^2_{S_0} C = 2\partial_{S_0} \Pi_1 + S_0 \partial^2_{S_0} \Pi_1 - K e^{-r\tau} \partial^2_{S_0} \Pi_2, \tag{2.40}$$

$$\text{Vega}_{v_0} = \partial_{v_0} C = S_0 \partial_{v_0} \Pi_1 - K e^{-r\tau} \partial_{v_0} \Pi_2, \tag{2.41}$$

$$\text{Rho} = \partial_r C = S_0 \partial_r \Pi_1 + K\tau e^{-r\tau} \Pi_2 - K e^{-r\tau} \partial_r \Pi_2, \tag{2.42}$$

$$\Theta = \partial_t C = -\partial_\tau C. \tag{2.43}$$

The practical advantage is that all sensitivities are expressed in terms of derivatives of the same characteristic-function integrands. In affine form, $\phi_X(u) = \exp(C(u, \tau) + D(u, \tau) v_0 + iu \log S_0)$, so parameter derivatives follow analytically from derivatives of C and D, and can be inserted directly into the integrals above.

2.6. Implied Volatility And Numerical Inversion

For each target option value V_{target} (market quote or model-generated price), implied volatility is obtained by inverting the Black–Scholes map [4]. This provides a common representation across strikes and maturities, which is the main comparison space used in this thesis.

Numerically, we solve the scalar nonlinear equation

$$F(\sigma) := V_{BS}(S_0, K, r, \tau, \sigma) - V_{\text{target}} = 0, \qquad \sigma > 0. \tag{2.44}$$

For European vanilla options, V_{BS} is strictly increasing in σ when $\tau > 0$, so the root is unique whenever V_{target} satisfies static no-arbitrage bounds.

The inversion routine used in this thesis is the bracketed Brent method [6, Ch. 9]. This choice is robust for the one-dimensional IV root and avoids derivative-sensitivity issues in low-vega regions. As an alternative, Levenberg–Marquardt was also tested in preliminary experiments, but it showed lower numerical stability in this workflow; therefore, it is not used in the final pipeline.

Stopping is based on standard price and parameter tolerances:

$$|F(\sigma_n)| \le \varepsilon_{\text{price}} \quad \text{or} \quad |\sigma_{n+1} - \sigma_n| \le \varepsilon_\sigma, \qquad n \le n_{\max}. \tag{2.45}$$

If convergence is not reached, or if the target price violates arbitrage bounds, the point is flagged as invalid for IV reporting. In edge cases near the IV floor, the pricing stage may be recomputed with a wider COS truncation interval before retrying inversion.

In the results chapters, IV errors are the primary evaluation metrics:

$$\text{MAE}_{IV} = \frac{1}{N}\sum_{i=1}^{N}\left|\hat{\sigma}_i - \sigma_i^{\text{ref}}\right|, \qquad \text{RMSE}_{IV} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\hat{\sigma}_i - \sigma_i^{\text{ref}}\right)^2}. \tag{2.46}$$

To capture tail behavior, quantiles of absolute IV error are also reported (e.g., q_{95} and q_{99}), together with stratified summaries by moneyness and maturity buckets.

2.7. State Of The Art

The literature relevant to this thesis can be grouped into three blocks. First, classical model-based pricing combines closed or semi-closed formulas (e.g., Black–Scholes and Heston) with deterministic numerical solvers under explicit stochastic assumptions [3], [4]. These methods are financially interpretable and robust, but repeated calls in calibration or surface-generation loops can be computationally expensive.

Second, spectral/Fourier pricing methods exploit characteristic functions to obtain fast and accurate valuation formulas, with FFT- and COS-based implementations as standard references [2], [7]. Their main strength is the accuracy-speed tradeoff in repeated pricing, while practical performance still depends on stable parameterization choices (truncation interval, series size, and numerical inversion settings).

Third, neural surrogates learn the pricing map directly from synthetic or market-consistent labels. Feed-forward ANN surrogates have shown substantial acceleration in calibration workflows [8]. More recently, PINN-type approaches have been proposed to

inject structural constraints and improve data efficiency [9], [10]. However, the PINN literature also reports optimization pathologies, sensitivity to loss balancing, and robustness issues outside the training distribution [11], [12], [13].

Within this landscape, this thesis reuses the Heston-COS framework as high-accuracy numerical reference and contributes an empirical comparison of data-driven alternatives (ANN and PINN-family variants) under a common data-generation, evaluation, and benchmarking protocol.

TABLE 2.2. METHODOLOGICAL BLOCKS AND ROLE IN THIS THESIS

Block	**Representative methods**	**Main strengths**	**Main limitations**
Classical model-based pricing	Black–Scholes, Heston semi-closed pricing [3], [4]	Financial interpretability; strong theoretical grounding	High computational cost in repeated evaluations
Spectral pricing	FFT/Carr–Madan, COS/Fang–Oosterlee [2], [7]	High accuracy with efficient repeated pricing	Requires careful numerical setup ($[a, b]$, N, inversion stability)
Data-driven surrogates	ANN pricers/calibrators [8]	Very fast inference once trained	Domain-shift and extrapolation risk
Physics-informed surrogates	PINN/gPINN variants [9], [10]	Injects structural constraints into learning	Training instability and sensitive optimization [11], [12], [13]

Source: Author's synthesis from the cited literature

3. NEURAL NETWORK PRICER FOR HESTON VANILLA OPTIONS

The main objective of this chapter is to construct an ANN-based surrogate pricer for European vanilla options under the Heston stochastic-volatility model. The target quantity is implied volatility, and the surrogate is designed to approximate the forward map

$$(\Theta, m, \tau, r) \mapsto \sigma_{imp}.$$

Reference pricing based on numerical methods (in our pipeline, Heston COS valuation plus implied-volatility inversion) is highly accurate but computationally expensive when repeated many times, as required in calibration and sensitivity workflows. The ANN aims to preserve pricing accuracy while reducing inference time by several orders of magnitude.

Methodologically, this chapter follows the ANN-pricing framework of Liu et al. [8] as the main reference (architecture family, data ranges, and training philosophy), while explicitly documenting the implementation choices adopted in this project.

This chapter focuses exclusively on the pricing network; the calibration architecture is treated separately in Chapter 4.

3.1. Problem Formulation

In this work, vanilla option pricing under Heston is posed as a supervised regression problem. Given a dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad y_i = \sigma_{imp,i},$$

the neural network approximates the map

$$f_{NN} : \mathbb{R}^8 \to \mathbb{R}, \qquad \mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \mapsto \sigma_{imp},$$

where $m = S_0/K$ is moneyness and $\tau = T - t$ is time-to-maturity. Each label σ_{imp} is obtained from Heston-consistent synthetic prices through numerical inversion.

The training objective is to estimate parameters $\mathbf{w}$ such that the empirical prediction error over $\mathcal{D}$ is minimized. In this chapter, we use the ANN only as a pricing surrogate, i.e., as a fast approximation of the reference numerical pipeline in the forward map $(\Theta, m, \tau, r) \mapsto \sigma_{imp}$.

We learn implied volatility instead of option price because the IV representation is directly comparable across contracts with different maturities and moneyness levels, and it is the market-standard quotation format for calibration and diagnostics. In contrast, raw

prices mix scale effects (spot/strike/time) that make learning less homogeneous over the domain.

The resulting surrogate is primarily an in-domain approximator. Preliminary out-of-domain checks indicate that the network remains stable for moderate departures from the training bounds, while errors increase in extreme regimes. For this reason, the methodological core of this chapter is developed on the predefined training domain, and the extrapolation behaviour is discussed separately in the generalization and robustness analysis.

3.2. Synthetic Data Formulation

Training requires a large set of labeled input-output pairs. To obtain a controlled and fully reproducible dataset, we use synthetic data generated under the Heston model. For each sampled input vector, the target implied volatility is computed through the numerical pricing pipeline (COS valuation followed by implied-volatility inversion).

In the current pipeline, data generation is organized as a *master dataset* shared across downstream models. Each row stores:

- features: $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$,
- targets: $(price_{\cos}, \sigma_{\text{imp}}^{\text{brent}})$,
- quality labels: $(feller\_slack, feller\_ok, brent\_success, brent\_abs\_residual)$.

This design allows ANN and PINN workflows to reuse the same synthetic source while applying experiment-specific filters only at training time.

The raw variable set includes Heston parameters $\Theta = \{\rho, \kappa, \gamma, \bar{v}, v_0\}$ and contract/market variables $\Sigma = \{K, S_0, \tau, r\}$, which defines a 9-dimensional space. In our setup, we fix $K = 1$ and represent the underlying level through moneyness $m = S_0/K$, leading to an 8-dimensional input vector:

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \in \mathbb{R}^8.$$

A full Cartesian grid is computationally prohibitive in this dimension, since using N points per variable scales as N^9 (already intractable for moderate N). Therefore, we rely on Latin Hypercube Sampling (LHS) to obtain broad coverage of the domain with a fixed number of samples. The specific ranges, constraints, and splitting/preprocessing rules are detailed in the following subsections.

3.2.1. Parameter Ranges And Constraints

The sampling domain combines Heston parameters, contract variables, and market rate. Following the ranges used in our configuration (largely aligned with [8]), the training

domain is summarized in the table below.

TABLE 3.1. INPUT RANGES USED FOR SYNTHETIC DATA GENERATION

Parameter	Range
ρ	$[-0.9, 0.0]$
κ	$[0.0, 3.0]$
γ	$[0.01, 0.8]$
$\bar{v}$	$[0.01, 0.5]$
v_0	$[0.05, 0.5]$
m	$[0.6, 1.4]$
τ	$[0.05, 3.0]$
r	$[0.0, 0.05]$

Source: Based on [8] and project configuration choices

To avoid non-physical or numerically unstable samples, we enforce $\tau > 0$ and monitor a Feller-type admissibility condition

$$2\kappa\bar{v} - \gamma^2 \geq 0,$$

through the scalar label

$$feller\_slack = 2\kappa\bar{v} - \gamma^2,$$

and the binary indicator $feller\_ok = \mathbb{1}\{feller\_slack \geq 0\}$. Importantly, Feller-violating points are *not* removed during generation; they are kept in the master dataset for controlled ablation studies.

During synthesis we fix the strike to $K = 1$ and price European put options. Fixing K is equivalent to expressing spot through moneyness, which reduces the dimensionality from nine variables $(\Theta, S_0, K, \tau, r)$ to eight inputs (Θ, m, τ, r) without loss of information for this setup.

3.2.2. Sampling Strategy: Latin Hypercube Sampling

The training of the neural network requires generating a large set of model parameters distributed over a high-dimensional domain. In our case, the parameter space is $\mathbb{R}^8$. A purely random sampling of this space may generate highly populated regions while leaving other regions scarcely explored. As a consequence, the resulting training dataset may be poorly representative, leading to suboptimal training performance.

To avoid this problem, we will use the *Latin Hypercube Sampling* (LHS), a sampling method designed to uniformly cover each dimension. The main idea consists in splitting each dimension of the parameter space into N disjoint subintervals of equal length, and randomly selecting exactly one sample within each subinterval.

Thus, LHS guarantees a one-dimensional marginal distribution along each coordinate, independently of the total dimension of the space. This results in a sampling scheme that is more representative of the parameter space.

This creates a representative distribution sampling of the parameter space. In contrast to a full-grid sampling, the total number of samples N remains fixed, preventing exponential growth in the number of dimensions.

The use of LHS is supported by the classical design-of-experiments literature, where stratified one-dimensional coverage is shown to improve space-filling behaviour with respect to naive random sampling for a fixed budget [14]. In our implementation, sampling is performed directly in the 8-dimensional input domain with a fixed seed and a target of 10^7 samples in the master dataset. This is consistent with standard surrogate-modeling practice, where the quality and coverage of the design points strongly influence final emulator accuracy and robustness [15].

3.2.3. Dataset Splitting And Preprocessing

After generation, we keep all selected rows in the master dataset and split into train/-validation/test subsets with proportions $0.8/0.1/0.1$. In the reported configuration, splits are random (without mandatory stratification), and filtering policies are deferred to each training experiment.

In the baseline configuration, no explicit feature or target normalization is applied (Liu-style setup). Therefore, the model is trained directly on the physical scale of the variables $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$ and predicts IV in absolute units.

Data quality control is integrated in the synthesis pipeline through explicit labels, not hard row deletion by default. In particular, we track Brent inversion status and reconstruction error

$$|V_{BS}(\sigma_{imp}) - V_{\text{target}}|,$$

while keeping all generated points available for later filtering. For storage efficiency, output tables are persisted in `float32`, whereas COS pricing and Brent inversion are computed internally in `float64`.

3.2.4. Numerical Label Generation (COS + Brent)

Each label is produced in two deterministic numerical steps. First, the target option value V_{target} is computed with the Heston COS solver for a given input vector $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$. Second, implied volatility is obtained by solving

$$V_{BS}(\sigma) = V_{\text{target}}$$

with a Brent root-finding routine under fixed bounds and tolerance [6].

In our implementation, Brent is configured with strict numerical controls (absolute tolerance 10^{-8}, bounded search interval, and iteration cap). Additionally, when the inferred volatility hits the lower IV floor, an adaptive COS fallback can be triggered: the truncation width is increased ($L \leftarrow 1.5L$, with optional N scaling) and inversion is retried. The retry is accepted only if it improves numerical quality (e.g., leaves the floor or reduces reconstruction error).

Rather than dropping those edge cases at generation time, the pipeline records quality metadata (`brent_success`, `brent_abs_residual`, and retry diagnostics) so that robustness filters can be applied transparently at training time. This preserves full traceability of numerical reliability across the domain.

3.3. Neural Network Architecture

The pricing surrogate is implemented as a fully connected neural network that approximates a smooth nonlinear map from model/contract inputs to implied volatility. This choice is consistent with the structure of the problem: inputs are low-dimensional continuous features without spatial topology or sequential dependence, and the target is a scalar regression value.

From a function-approximation perspective, a feed-forward multilayer perceptron (MLP) provides enough expressive capacity to capture the interactions between Heston parameters, moneyness, maturity, and rates on the selected domain. Compared with architectures designed for images or sequences (e.g., CNNs or transformers), an MLP is more parsimonious and easier to train for this tabular setting.

This choice is also consistent with the classical universal-approximation results: under standard assumptions, feed-forward networks with non-polynomial activations can approximate continuous functions on compact sets to arbitrary accuracy [16], [17]. In this thesis, this theorem is used as a representation-capacity motivation, while practical accuracy is established empirically through out-of-sample benchmarks.

3.3.1. Model Class

Following [8], we use a fully connected feed-forward network (FCNN). Let $\mathbf{x} \in \mathbb{R}^8$ be

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r),$$

and let $y = \sigma_{imp} \in \mathbb{R}$ denote the target implied volatility. The model defines a parametric function

$$\hat{\sigma}_{imp} = f_{NN}(\mathbf{x}; \mathbf{w}),$$

with trainable parameters $\mathbf{w}$.

Once $\mathbf{w}$ is fixed after training, inference is deterministic: for the same input $\mathbf{x}$, the network always returns the same prediction $\hat{\sigma}_{imp}$. This property is important for downstream

calibration and sensitivity analysis, where numerical consistency across repeated calls is required.

The FCNN acts as a surrogate only at inference time. Target labels are still generated with the numerical pipeline (Heston COS pricing followed by implied-volatility inversion), so the network learns to emulate that reference solver on the sampled domain.

3.3.2. Architecture Details

The architecture used in this chapter is

$$8 \to 200 \to 200 \to 200 \to 200 \to 1,$$

with ReLU activation in hidden layers and linear activation in the output layer. In compact form,

$$h^{(1)} = \phi(W^{(1)}x + b^{(1)}), \quad \ldots, \quad h^{(4)} = \phi(W^{(4)}h^{(3)} + b^{(4)}), \quad \hat{y} = W^{(5)}h^{(4)} + b^{(5)},$$

where $\phi(\cdot) = \max(0, \cdot)$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled ($p = 0$) and no batch normalization layers are used. This configuration follows a controlled, reproducible design intended to isolate the impact of optimization and data quality choices.

TABLE 3.2. NEURAL NETWORK ARCHITECTURE USED FOR THE ANN PRICER

Hyperparameter	**Value**
Input dimension	8
Hidden layers	4
Hidden width	200 neurons per layer
Hidden activation	ReLU
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Based on [8]

3.3.3. Output Interpretation

The network output is a scalar implied volatility in annualized units (decimal form), not an option price. Therefore, prediction errors are interpreted directly in IV space, which is the representation used in the calibration workflow and in most volatility-surface diagnostics.

Because the output layer is linear, the model is not hard-constrained to produce strictly positive values for every possible input. In practice, training on a physically admissible domain with positive IV labels strongly reduces this risk inside the sampled region. For downstream use, any out-of-domain prediction can be explicitly checked and, if required, filtered before subsequent numerical steps.

This output definition also preserves differentiability of the surrogate with respect to inputs, which is useful in later chapters when the ANN pricer is embedded in calibration and sensitivity pipelines.

3.4. Training Procedure

Training is fully configuration-driven and follows an epoch-based supervised learning loop. At each epoch, the active optimizer performs parameter updates on the training split, then the model is evaluated on the validation split without gradient computation. For Adam updates, optimization is done with standard mini-batch backpropagation; for L-BFGS updates, each step is computed through a closure that evaluates the loss and its gradient on the selected batch regime.

Each epoch stores train/validation metrics, current optimizer identity, and learning rate. When enabled, the validation monitor can be either the raw validation loss or a trimmed variant that discards the top-tail hardest samples before aggregation. This provides a robust alternative when a small fraction of outlier points may dominate the monitor value.

Execution device is selected automatically (`mps` when available, otherwise `cpu`), and data-shuffling reproducibility is controlled through a fixed generator seed from configuration. During training, two checkpoints are maintained: the *last* checkpoint (always overwritten each epoch) and the *best* checkpoint (updated only when the monitored validation metric improves). The best checkpoint is used as the reference model for post-training evaluation and downstream calibration.

3.4.1. Loss Function

Let $\{(\mathbf{x}_i, \sigma_i)\}_{i=1}^{B}$ denote a mini-batch, where σ_i is the target implied volatility and $\hat{\sigma}_i = f_{NN}(\mathbf{x}_i; \mathbf{w})$. The baseline objective is mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}}(\mathbf{w}) = \frac{1}{B} \sum_{i=1}^{B} (\hat{\sigma}_i - \sigma_i)^2 .$$

MSE is preferred because it is smooth, differentiable everywhere, and strongly penalizes large deviations, which is desirable when the ANN is later used in calibration loops sensitive to local pricing errors. Although MAE and RMSE are available in the implementation, MSE is used as the main training criterion in the reported baseline.

3.4.2. Optimizer And Hyperparameters

PUEDE IR CAMBIANDO: se pretende mejorar las runs

Several optimizer configurations were tested (Adam, mixed schedules, and `mix_half`). For model selection, we prioritize out-of-sample absolute-error behaviour (MAE and quantile stability), since these metrics better reflect the typical pointwise error level used later in calibration and sensitivity tasks.

Under this criterion, the selected reference ANN is `Liu_like_v01` (Adam), while `MIX_half_v01` is the second-best candidate. Although `MIX_half_v01` achieves a slightly lower validation MSE, `Liu_like_v01` provides lower validation and test MAE, together with better high-quantile absolute errors.

TABLE 3.3. REFERENCE AND RUNNER-UP RUNS USED IN OPTIMIZER SELECTION

Run	Optimizer mode	Val MAE	Test MAE
`Liu_like_v01`	Adam	4.1132×10^{-4}	4.2925×10^{-4}
`MIX_half_v01`	Adam → L-BFGS	5.1342×10^{-4}	5.2377×10^{-4}

Source: Own elaboration from `outputs/runs/*/metrics/eval_global.csv`

The hyperparameters of the selected reference run (`Liu_like_v01`) are summarized below.

TABLE 3.4. TRAINING HYPERPARAMETERS OF THE SELECTED RUN (`LIU_LIKE_V01`)

Hyperparameter	Value
Training mode	Adam
Seed (data-loader generator)	42
Epochs	8000
Train batch size	1024
Validation batch size	All validation samples
Loss	MSE
Adam learning rate	10^{-3}
Adam weight decay	0.0
LR scheduler	StepLR, step_size = 500, $\gamma = 0.5$
Early stopping	Disabled

Source: Own elaboration from run configuration copies

3.4.3. Regularization And Callbacks

PUEDE IR CAMBIANDO: se pretende mejorar las runs

Regularization in the baseline is intentionally light: dropout is disabled ($p = 0$), batch normalization is disabled, and Adam weight decay is set to zero. In addition, feature/target normalization is turned off in the Liu-style reference setup. This design keeps the training protocol close to the reference architecture and isolates the effect of optimizer scheduling and data quality controls.

Callback management includes: (i) checkpointing of both *best* and *last* models, (ii) StepLR scheduling for Adam, and (iii) an early-stopping module available in the pipeline but disabled in the baseline experiments. When enabled, early stopping supports both standard validation loss and trimmed validation loss monitors.

From an experimental standpoint, this callback configuration improves training robustness without over-constraining the optimization path: checkpoints protect against late-epoch degradation, while the Adam-to-L-BFGS transition plus scheduler supports stable convergence. Generalization is then assessed empirically on held-out validation and test splits rather than enforced through strong explicit regularizers.

4. CALIBRATION NEURAL NETWORK FOR THE HESTON MODEL

4.1. Calibration problem formulation

- hablar sobre el concepto de generalization
- hablar sobre Differential Evolution. asi como sus argumentos e interpretación (en Liu está bien explicado)

4.2. Input representation: implied volatility surfaces

4.3. Synthetic calibration dataset

4.4. CaNN architecture

5. PHYSICS-INFORMED NEURAL PRICING UNDER THE HESTON MODEL

This chapter reformulates vanilla option pricing under the Heston model as a PDE-constrained learning problem. In contrast to the supervised ANN surrogate of Chapter 3, the objective is no longer to emulate precomputed numerical labels, but to approximate directly the option-price function through an unsupervised Physics-Informed Neural Network (PINN). The methodology follows the general PINN philosophy and is inspired by the Heston-specific formulation proposed in [9], while adapting the state representation and notation to the conventions used in the present work.

5.1. Problem formulation

In this chapter, European option pricing under Heston is posed as the approximation of the solution of the pricing PDE rather than as a supervised regression problem. The neural network outputs the option price itself, and the training objective is to enforce that this output satisfies the Heston valuation equation together with its terminal and boundary conditions. Consequently, the core information driving the optimization is not a set of external labels, but the analytical structure of the model under the risk-neutral measure.

To avoid a notational clash between the price function and the variance state, the option value will be denoted by u, while the instantaneous variance state will be denoted by v. This distinction is important because the Heston model contains both a variance process and a pricing function, and using the same symbol for both would make the PDE formulation unnecessarily confusing. The unknown function approximated by the PINN is therefore written as

$$u_\phi(\tau, m, v, \Theta, r),$$

where ϕ denotes the trainable weights of the network, $m = S/K$ is moneyness, $\tau = T - t$ is time-to-maturity, v is the current instantaneous variance, and Θ collects the structural Heston parameters.

More precisely, the calibrated tuple obtained in the previous chapter is

$$\Theta^* = (\rho, \kappa, \gamma, \bar{v}, v_0).$$

However, the role of v_0 must be distinguished from that of the other parameters. The quantities ρ, κ, γ and $\bar{v}$ define the operator of the Heston PDE, whereas v_0 is the initial condition of the variance process. In the PDE formulation, the state variable is the generic variance level v, not the fixed initial value v_0. Therefore, the parametric PINN is constructed to learn a family of Heston pricing solutions indexed by the structural parameters $(\rho, \kappa, \gamma, \bar{v}, r)$, while the current variance v_0 is used only when evaluating the trained

network at the present market state. The calibrated market valuation is thus recovered through

$$u_\phi(\tau, m, v_0, \rho^*, \kappa^*, \gamma^*, \bar{v}^*, r).$$

This point is conceptually important: during training, the PINN learns the solution over a domain in $(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$, whereas at inference time the current market valuation is recovered by fixing the variance coordinate to the calibrated value v_0^* and the structural parameters to the calibrated tuple Θ^*.

The network is trained on collocation points sampled in the interior and on the relevant boundaries of the pricing domain. These points are not generated by simulating full paths of the Heston variance process. Instead, they are drawn directly from a bounded domain of admissible states, for instance using Latin Hypercube Sampling (LHS), and used to evaluate the PDE residual. In that sense, the method is unsupervised: the learning signal comes from the Heston structure itself, not from externally computed price labels. This is the main methodological difference with respect to the ANN surrogate developed in Chapter 3.

At a high level, the proposed workflow is the following. First, a parametric PINN is defined to approximate the price surface over a range of maturities, moneyness levels, variance states, and Heston parameters. Second, the PINN is trained by minimizing the residual of the Heston pricing PDE together with terminal and boundary mismatches. Third, once the network is trained, the price of a contract for a calibrated market configuration is obtained by evaluating the network at the corresponding $(\tau, m, v_0^*, \Theta^*, r)$. This preserves the link with the calibration pipeline while replacing label-based supervision with model-consistent unsupervised training. In this sense, the CaNN stage remains essential: it provides the market-specific parameter tuple that is consumed by the parametric PINN at inference time, without requiring a retraining of the pricing network for each new calibration.

- hablar muy bien de los inputs (heston params optimizados y tal)
- hablar sobre el output, que es V para vanilla option
- hablar sobre greeks por autodiff ?
- hablar de nuevo un poco sobre el pipeline

5.1.1. PINN Theoretical Framework

no estoy seguro de donde poner esto, o si quitarlo incluso

A PINN can be viewed as a parametric approximation of the solution of a differential equation. In the present setting, the network u_ϕ is required to satisfy the Heston pricing PDE in the interior of the domain and to match the contractual conditions at expiry and

on selected boundaries. If $\mathcal{N}$ denotes the Heston differential operator, the target problem can be summarized abstractly as

$$\mathcal{N}[u] = 0 \quad \text{in the interior domain,} \qquad \mathcal{B}[u] = 0 \quad \text{on the boundaries,}$$

where $\mathcal{B}$ denotes the set of terminal and boundary constraints.

The PINN replaces the unknown analytical solution by a neural approximation u_ϕ and defines a residual function

$$\mathcal{R}_\phi = \mathcal{N}[u_\phi].$$

Training then consists of minimizing a composite loss built from three sources of error: the interior PDE residual, the mismatch with the terminal payoff, and the mismatch with the selected boundary conditions. In a compact notation, the loss takes the form

$$\mathcal{L}_{\text{PINN}}(\phi) = \mathcal{L}_{\text{PDE}}(\phi) + \mathcal{L}_{\text{term}}(\phi) + \mathcal{L}_{\text{bc}}(\phi).$$

This decomposition is standard in the PINN literature and matches the structure adopted in [9].

An essential ingredient of this approach is automatic differentiation. Since the Heston PDE contains first- and second-order derivatives with respect to the state variables, the network output must remain differentiable with respect to its inputs. The required quantities are obtained by differentiating the computational graph of the network rather than by using finite-difference approximations. This is particularly convenient in the present case because the same differentiable surrogate can later be reused for the computation of Greeks.

- función aproximadora
- idea de residuo de la PDE
- concepto de la loss en una PINN

5.2. Heston Pricing PDE

Under the risk-neutral measure, the Heston model is described by the coupled system dejar solo en los primeros chapters y referencialos, o poner bibliografia

$$dS_t = rS_t\,dt + \sqrt{v_t}\,S_t\,dW_t^{(1)}, \tag{5.1}$$
$$dv_t = \kappa(\bar{v} - v_t)\,dt + \gamma\sqrt{v_t}\,dW_t^{(2)}, \tag{5.2}$$

with instantaneous correlation

$$d\langle W^{(1)}, W^{(2)}\rangle_t = \rho\,dt.$$

Here r is the risk-free rate, κ is the mean-reversion speed of the variance process, $\bar{v}$ is the long-run variance level, γ is the volatility of variance, and ρ controls the correlation

between returns and variance shocks. The current variance level at valuation time is denoted by v_0, so that v_t evolves from the initial condition $v(0) = v_0$.

Let $u(t, S, v)$ denote the price of a European vanilla option. By the Feynman-Kac representation, u satisfies the Heston pricing PDE [3]

$$u_t + \frac{1}{2}vS^2u_{SS} + \rho\gamma vSu_{Sv} + \frac{1}{2}\gamma^2 vu_{vv} + rSu_S + \kappa(\bar{v} - v)u_v - ru = 0. \tag{5.3}$$

This is the continuous-time constraint that will be enforced by the PINN at interior collocation points. In the present chapter, the price function is denoted by u precisely to keep it separate from the variance state v.

For implementation purposes, it is more convenient to work with time-to-maturity $\tau = T - t$ and moneyness $m = S/K$ instead of calendar time t and spot S. Since the synthetic and market pipelines of this project use a fixed strike normalization $K = 1$, the pricing problem can be rewritten in terms of the state variables (τ, m, v). Defining the same pricing function in transformed coordinates, the PDE becomes

$$u_\tau = \frac{1}{2}vm^2u_{mm} + \rho\gamma vmu_{mv} + \frac{1}{2}\gamma^2 vu_{vv} + rmu_m + \kappa(\bar{v} - v)u_v - ru. \tag{5.4}$$

This is the formulation that will be used to define the PDE residual in the training loss. It remains fully equivalent to the original Heston pricing equation, but it is better aligned with the representation used in the rest of the thesis.

For a European put option with normalized strike $K = 1$, the terminal condition at expiry is

$$u(0, m, v) = (1 - m)^+. \tag{5.5}$$

The payoff does not depend explicitly on the variance state, which means that the terminal condition is imposed uniformly over the v-dimension of the domain. In addition, a natural lower boundary condition is obtained at zero underlying value:

$$u(\tau, 0, v) = e^{-r\tau}. \tag{5.6}$$

For a general strike K, this condition would read $u(\tau, 0, v) = Ke^{-r\tau}$. In a first PINN specification, these two constraints are sufficient to reproduce the structure used in [9]: one term of the loss is attached to the interior PDE residual, one to the terminal payoff, and one to the lower boundary.

The far-field behavior as $m \to \infty$ for a put, namely $u(\tau, m, v) \to 0$, can also be incorporated if needed. However, to keep the initial implementation simple, the first version of the model will explicitly enforce only the terminal condition and the lower boundary, while controlling the remaining behavior through the bounded sampling domain in (τ, m, v) and the interior residual. This choice is consistent with the objective of building a minimal but coherent unsupervised PINN before adding more elaborate constraints.

- presentar la PDE de heston y hablar sobre ella

- definir la terminal condition payoff (depende de put/call !)
- hablar de las boundary conditions y cuales imponemos

5.3. Neural Network Architecture

5.3.1. Model Class

The pricing PINN is implemented as a fully connected feed-forward neural network (MLP) that approximates the mapping

$$u_\phi : \mathbb{R}^8 \to \mathbb{R}, \qquad x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r) \mapsto u_\phi(x),$$

where the scalar output is the option price. This architecture is appropriate for the present setting because all inputs are continuous tabular variables and no spatial or sequential inductive bias is required. citar referencia general sobre MLPs para PINNs

Compared with Chapter 3, where the ANN surrogate learned implied volatility from supervised labels, the role of the network here is to represent a differentiable pricing function that is constrained by the Heston PDE and boundary conditions. The model class is therefore selected not only for approximation capacity, but also for stable first- and second-order automatic differentiation. citar referencia sobre el papel de la diferenciabilidad en PINNs

5.3.2. Architecture Details

The reference architecture used in the implementation is

$$8 \to 256 \to 256 \to 256 \to 256 \to 1,$$

with `tanh` activation in hidden layers and linear activation in the output layer. In compact notation,

$$h^{(1)} = \varphi(W^{(1)}x + b^{(1)}), \ \ldots, \ h^{(4)} = \varphi(W^{(4)}h^{(3)} + b^{(4)}), \quad u_\phi = W^{(5)}h^{(4)} + b^{(5)},$$

with $\varphi = \tanh$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled and no batch-normalization layers are included. FIGURE: diagrama de arquitectura PINN (8-256-256-256-256-1, activaciones `tanh`, salida lineal)

TABLE 5.1. PINN ARCHITECTURE USED IN THIS CHAPTER

Hyperparameter	**Value**
Input dimension	8
Hidden layers	4
Hidden width	256 neurons per layer
Hidden activation	`tanh`
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Own implementation (`configs/pinn_model_architecture.yaml`)

5.3.3. Model Inputs and Outputs

The model receives the following ordered input vector:

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r),$$

where (τ, m, v) are state variables, r is the risk-free rate, and $(\rho, \kappa, \gamma, \bar{v})$ are the structural Heston parameters that define the PDE operator. TABLE: resumen de inputs del PINN (variable, significado financiero, rol en PDE, rango de entrenamiento)

The network output is a scalar price $u_\phi(x)$, corresponding to the normalized-strike vanilla contract considered in this chapter ($K = 1$). This output choice is consistent with the PDE-constrained formulation: the PINN directly approximates the pricing function rather than an implied-volatility transformation. FIGURE: esquema de entradas/salida y conexión CaNN → PINN en inferencia

In the current experimental setup, collocation generation is run in a fixed-parameter regime for $(\rho, \kappa, \gamma, \bar{v})$, while (τ, m, v, r) vary across points. This means the model remains formally 8-dimensional, but some coordinates may be constant in a given run due to calibration-conditioned sampling. aclarar si este apartado se mantiene como baseline fixed-theta o si se añade variante fully parametric

Input Scaling

Although the pricing problem is formulated in physical variables, the optimization is performed with an affine rescaling of the network inputs. The motivation is numerical: (τ, m, v) and the Heston-related coordinates do not share comparable magnitudes, which can deteriorate conditioning and gradient balance. Following practical PINN recommen-

dations, centering and scaling are used as a stabilization device. citar referencia sobre scaling en PINNs

In the implemented pipeline, the transformation is applied componentwise to the full PINN input vector through

$$\tilde{x}_d = a_d + b_d x_d, \qquad d = 1, \dots, 8.$$

The coefficients are estimated from the collocation points of the training split:

$$\mu_d = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} x_{i,d}, \qquad \sigma_d = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (x_{i,d} - \mu_d)^2},$$

and then

$$a_d = -\frac{\mu_d}{\sigma_d}, \qquad b_d = \frac{1}{\sigma_d}.$$

To avoid numerical issues on nearly constant coordinates, the implementation uses a small threshold $\varepsilon = 10^{-8}$: if $\sigma_d \le \varepsilon$, that coordinate is left unscaled ($a_d = 0,\ b_d = 1$).

The PDE residual is still defined in physical variables and differentiated by automatic differentiation. In code, the network is evaluated at $\tilde{x}$, while derivatives are taken with respect to the original coordinates x; the chain rule is therefore handled by the computational graph. Consequently, input scaling changes the numerical conditioning of training, but not the financial meaning of the PDE constraint. FIGURE: comparación visual antes/después del escalado en los inputs principales

5.4. Training Procedure

5.4.1. Loss Function

The PINN is trained by minimizing a composite loss that enforces the pricing PDE in the interior of the domain together with the terminal payoff and the lower boundary condition. Let $u_\phi(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$ denote the output of the neural network. From the transformed Heston PDE introduced above, the residual at an interior point is defined as

$$\mathcal{R}_\phi = u_\tau - \frac{1}{2} v m^2 u_{mm} - \rho\gamma v m u_{mv} - \frac{1}{2}\gamma^2 v u_{vv} - r m u_m - \kappa(\bar{v} - v) u_v + r u, \tag{5.7}$$

where all derivatives are evaluated by automatic differentiation on the computational graph of the network. In the ideal case, the exact pricing function would satisfy $\mathcal{R}_\phi = 0$ for every admissible interior point.

Let $\mathcal{S}_D = \{z_j\}_{j=1}^{N_D}$ denote the set of interior collocation points, with

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j).$$

The interior loss is the mean squared residual

$$\mathcal{L}_{\text{PDE}}(\phi) = \frac{1}{N_D} \sum_{j=1}^{N_D} \mathcal{R}_\phi(z_j)^2. \tag{5.8}$$

On the terminal set $\mathcal{S}_T = \{z_k^T\}_{k=1}^{N_T}$, where $\tau = 0$, the network must match the payoff of the European put. The corresponding error is

$$e_k^T(\phi) = u_\phi(0, m_k, v_k, \rho_k, \kappa_k, \gamma_k, \bar{v}_k, r_k) - (1 - m_k)^+, \tag{5.9}$$

and the terminal loss is defined by

$$\mathcal{L}_{\text{term}}(\phi) = \frac{1}{N_T} \sum_{k=1}^{N_T} \left(e_k^T(\phi)\right)^2. \tag{5.10}$$

On the lower boundary set $\mathcal{S}_{\text{low}} = \{z_l^{\text{low}}\}_{l=1}^{N_{\text{low}}}$, where $m = 0$, the exact put price is the discounted strike. Since the implementation uses the normalized strike $K = 1$, the lower-boundary error is

$$e_l^{\text{low}}(\phi) = u_\phi(\tau_l, 0, v_l, \rho_l, \kappa_l, \gamma_l, \bar{v}_l, r_l) - e^{-r_l \tau_l}, \tag{5.11}$$

and the associated loss is

$$\mathcal{L}_{\text{bc}}(\phi) = \frac{1}{N_{\text{low}}} \sum_{l=1}^{N_{\text{low}}} \left(e_l^{\text{low}}(\phi)\right)^2. \tag{5.12}$$

The total training objective is then written as

$$\mathcal{L}_{\text{PINN}}(\phi) = \mathcal{L}_{\text{PDE}}(\phi) + \mathcal{L}_{\text{term}}(\phi) + \mathcal{L}_{\text{bc}}(\phi). \tag{5.13}$$

For implementation, it is convenient to keep an explicit weighted form,

$$\mathcal{L}_{\text{PINN}}(\phi) = \lambda_{\text{PDE}} \mathcal{L}_{\text{PDE}}(\phi) + \lambda_{\text{term}} \mathcal{L}_{\text{term}}(\phi) + \lambda_{\text{low}} \mathcal{L}_{\text{bc}}(\phi), \tag{5.14}$$

with non-negative coefficients $(\lambda_{\text{PDE}}, \lambda_{\text{term}}, \lambda_{\text{low}})$ that can be tuned if one component dominates the optimization. In the baseline specification of this thesis, all three weights are initialized to one, i.e. $(1, 1, 1)$, following the simplest formulation in [9].

A practical point is that the PDE term requires second-order derivatives with respect to inputs. Therefore, the automatic-differentiation graph must be kept when computing first derivatives, so that mixed and second derivatives such as u_{mv} and u_{vv} can be obtained consistently in the same computational graph. This is the key implementation detail that enables direct optimization of the PDE residual without finite-difference approximations.

5.4.2. Optimizer and Hyperparameters

The optimization follows a two-stage strategy that is common in PINN practice: an initial stochastic phase with Adam, followed by a quasi-Newton refinement with L-BFGS. In this work, this strategy is implemented as a mixed schedule in which Adam is used during the first part of training and L-BFGS in the second part, while a pure Adam or pure L-BFGS run remains available as an ablation.

Since the three loss terms are evaluated on different sample sets, each epoch uses three mini-batches: one from $\mathcal{S}_D$ (interior), one from $\mathcal{S}_T$ (terminal), and one from $\mathcal{S}_{\text{low}}$ (lower boundary). This allows different batch sizes for interior and boundary constraints and keeps the training loop aligned with the composite objective.

The reference hyperparameter block for the first implementation includes: learning rates and optimizer-specific settings, the Adam-to-L-BFGS switching epoch in mixed mode, batch sizes for interior and boundary sets, and the triplet of loss weights $(\lambda_{\text{PDE}}, \lambda_{\text{term}}, \lambda_{\text{low}})$.

For diagnostics, the training run stores epoch-wise train/validation curves for the total weighted loss and for each component separately. In addition, a two-dimensional error map over (m, τ) is produced from validation interior points by averaging $|\mathcal{R}_\phi|$ in bins. This map helps identify whether specific regions of the contract domain remain systematically underfitted even when the global loss decreases.

5.4.3. Sampling strategy

Training requires three different sample sets, each associated with one component of the loss. As in the general PINN methodology, these sets are not built from labeled prices but from points drawn directly in the state-parameter domain. The first set corresponds to the interior of the PDE domain, the second to the terminal boundary at expiry, and the third to the lower boundary at zero underlying value.

The interior domain is defined over bounded intervals for the state variables and for the Heston parameters:

$$\tau \in [0, \tau_{\max}], \qquad m \in [m_{\min}, m_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

combined with admissible ranges for $(\rho, \kappa, \gamma, \bar{v}, r)$. These parameter intervals should remain consistent with the calibration regime studied in the previous chapters, since the purpose of the parametric PINN is precisely to generalize over the family of Heston configurations that may be produced by the CaNN stage. In particular, the v-dimension is sampled explicitly as a state coordinate of the PDE. This is a crucial point: the PINN is trained on generic variance states v, whereas the calibrated initial variance v_0 is only used later, when evaluating the trained network at the current market state.

To obtain a representative coverage of this multidimensional domain with a controlled number of points, the baseline sampling method will be Latin Hypercube Sampling (LHS). Compared with naive Monte Carlo sampling, LHS provides a more regular coverage of each marginal dimension and reduces the risk of leaving large regions of the domain underexplored. In the present setting, this is particularly useful because the PINN must generalize simultaneously over state variables and model parameters.

For the interior set $\mathcal{S}_D$, each sampled point has the form

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j), \qquad j = 1, \ldots, N_D.$$

These are the collocation points at which the PDE residual is evaluated. For the terminal set $\mathcal{S}_T$, points are sampled in the subdomain

$$\tau = 0, \qquad m \in [m_{\min}, m_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

again jointly with the Heston parameters. This set is used to impose the payoff condition. For the lower-boundary set $\mathcal{S}_{\text{low}}$, the sampled points satisfy

$$m = 0, \qquad \tau \in [0, \tau_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

with the same parameter ranges, and are used to enforce the discounted-strike boundary condition.

From an implementation viewpoint, the three sets can be regenerated dynamically during training or precomputed once and reused across epochs. A dynamic resampling strategy may improve coverage of the domain and reduce overfitting to a fixed cloud of collocation points, whereas a static design simplifies reproducibility and debugging. The initial implementation will prioritize simplicity and reproducibility; therefore, the first reference version will rely on fixed sample sets generated before the training loop. Dynamic resampling can later be evaluated as an extension if it leads to more stable convergence.

This sampling strategy preserves the fully unsupervised nature of the method. No prices from COS, FFT, or Monte Carlo are required to build the training objective. Numerical pricing methods remain useful for ex-post validation and benchmarking, but not as a source of labels for fitting the PINN itself.

5.5. Extension: Sobolev-Enhanced Training for Greek Accuracy

The baseline PINN objective in this chapter is designed to recover prices by enforcing the Heston PDE and boundary/terminal conditions. However, accurate prices do not automatically imply accurate first- and second-order sensitivities. In particular, Γ and variance/rate sensitivities may show larger local errors because they depend on curvature properties of the learned surface. To address this limitation, a natural extension is to enrich training with derivative-aware terms inspired by Sobolev training [18] and gradient-enhanced PINN formulations [10].

The idea is to preserve the original physics loss and add a controlled supervision signal on selected Greeks at anchor points. Let

$$\mathcal{L}_{\text{base}} = \lambda_{\text{pde}}\mathcal{L}_{\text{pde}} + \lambda_{\text{term}}\mathcal{L}_{\text{term}} + \lambda_{\text{low}}\mathcal{L}_{\text{low}},$$

and let $\mathcal{S}_{\text{g}}$ be a set of Greek-anchor points in the same input space

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

For each $x \in \mathcal{S}_g$, reference sensitivities $\mathcal{G}^\star(x)$ are obtained from the semi-analytical Heston characteristic-function map (Section 2.5.2), while $\mathcal{G}_\phi(x)$ are computed from autodiff on the PINN output.

Using normalized residuals to avoid scale dominance across Greeks, we define

$$\mathcal{L}_{g,1} = \frac{1}{|\mathcal{S}_g|} \sum_{x \in \mathcal{S}_g} \sum_{q \in \{\Delta, \text{Vega}_v, \Theta, \text{Rho}\}} \left(\frac{q_\phi(x) - q^\star(x)}{s_q + \varepsilon} \right)^2, \tag{5.15}$$

$$\mathcal{L}_{g,2} = \frac{1}{|\mathcal{S}_g|} \sum_{x \in \mathcal{S}_g} \left(\frac{\Gamma_\phi(x) - \Gamma^\star(x)}{s_\Gamma + \varepsilon} \right)^2, \tag{5.16}$$

where s_q are fixed scale factors (e.g., robust quantiles from the reference sensitivities) and $\varepsilon > 0$ avoids numerical singularities near zero.

The prototype objective is then

$$\mathcal{L}_{\text{proto}} = \mathcal{L}_{\text{base}} + \lambda_{g,1}\mathcal{L}_{g,1} + \lambda_{g,2}\mathcal{L}_{g,2}. \tag{5.17}$$

From an optimization viewpoint, this extension introduces additional multi-term imbalance risks, already documented in the PINN literature [11], [12], [13]. Therefore, the intended training strategy is two-stage: first, train the baseline PINN to a stable price solution; second, apply a short fine-tuning phase with gradually increased $(\lambda_{g,1}, \lambda_{g,2})$, while monitoring possible regressions in price/PDE metrics.

This section defines a methodological prototype for subsequent work. It is included to fix the exact formulation to be tested, but it is not yet part of the benchmarked implementation reported in this thesis.

5.6. Greeks Computation

5.6.1. Definition of Greeks from the PINN Output

Once the PINN has been trained, the network output is a differentiable approximation of the vanilla option price:

$$u_\phi(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

Therefore, sensitivities can be obtained directly as partial derivatives of this function, evaluated at a chosen market point. Let

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

The first-order and second-order derivatives with respect to the input coordinates are denoted by

$$\nabla_x u_\phi(x), \qquad \nabla_x^2 u_\phi(x).$$

From these objects, standard trading Greeks are extracted as follows:

$$\Delta = \frac{\partial u_\phi}{\partial S}, \tag{5.18}$$

$$\Gamma = \frac{\partial^2 u_\phi}{\partial S^2}, \tag{5.19}$$

$$\Theta = \frac{\partial u_\phi}{\partial t} = -\frac{\partial u_\phi}{\partial \tau}, \tag{5.20}$$

$$\text{Rho} = \frac{\partial u_\phi}{\partial r}. \tag{5.21}$$

In this work, the spatial coordinate is moneyness $m = S/K$, not spot directly. Hence a chain-rule conversion is required:

$$\frac{\partial u_\phi}{\partial S} = \frac{\partial u_\phi}{\partial m}\frac{\partial m}{\partial S} = \frac{1}{K}\frac{\partial u_\phi}{\partial m}, \tag{5.22}$$

$$\frac{\partial^2 u_\phi}{\partial S^2} = \frac{1}{K^2}\frac{\partial^2 u_\phi}{\partial m^2}. \tag{5.23}$$

For the normalized-strike setup used in the thesis ($K = 1$), both coordinate systems coincide numerically.

Since the PINN state includes instantaneous variance v, a natural model-consistent volatility sensitivity is

$$\text{Vega}_v := \frac{\partial u_\phi}{\partial v}.$$

If needed, this can be transformed to a sensitivity with respect to volatility level $\sigma_v = \sqrt{v}$:

$$\frac{\partial u_\phi}{\partial \sigma_v} = 2\sqrt{v}\,\frac{\partial u_\phi}{\partial v}.$$

In addition, the parametric PINN provides direct structural sensitivities

$$\frac{\partial u_\phi}{\partial \rho}, \quad \frac{\partial u_\phi}{\partial \kappa}, \quad \frac{\partial u_\phi}{\partial \gamma}, \quad \frac{\partial u_\phi}{\partial \bar{v}},$$

which are useful for risk decomposition and calibration diagnostics beyond classical desk Greeks.

5.6.2. Automatic Differentiation of the PINN

The derivative computation is based on automatic differentiation over the computational graph of the trained network. For a single input point x, we compute:

$$u_\phi(x), \qquad J(x) = \nabla_x u_\phi(x), \qquad H(x) = \nabla_x^2 u_\phi(x).$$

Because the model is differentiable end-to-end, these quantities are exact derivatives of the neural surrogate (up to floating-point precision), not finite-difference approximations.

Compared with bump-and-reprice finite differences, this approach has three practical advantages in the PINN context:

- it avoids step-size tuning and the associated truncation/cancellation trade-off;
- it provides first- and second-order sensitivities in a single coherent graph;
- it scales efficiently to batched inputs through vectorized Jacobian/Hessian evaluation.

From a financial perspective, the interpretation remains standard: each Greek is the local slope or curvature of the price surface produced by the PINN around the evaluation point. The quality of those sensitivities therefore depends on both the local smoothness of the trained network and the fidelity of the learned price function in that region.

6. NEURAL NETWORK PERFORMANCE AND BENCHMARKING

6.1. Experimental Setup And Evaluation Protocol

This chapter compares the three model blocks developed in the thesis under a common evaluation protocol: ANN pricing surrogate, calibration network workflow, and PINN-based pricing/Greeks.

Unless explicitly stated, all reported metrics are computed on fixed evaluation sets with identical feature conventions, consistent preprocessing, and the same numerical references used in previous chapters.

The minimal pre-benchmark validation protocol includes:

- convergence checks with respect to (N, L) in the COS expansion,
- put-call parity residual checks across strikes,
- Black–Scholes consistency checks in near-constant-variance regimes.

Add final table with dataset/split definitions, run identifiers, and hardware context used for all benchmarks in this chapter.

6.2. ANN Pricer Performance

This section reports the performance of the supervised ANN pricer introduced in Chapter 3. The analysis is organized around implied-volatility error, regional error behavior over (m, τ), and inference cost under batched evaluation.

Insert ANN global metrics table (MAE/RMSE/quantiles) from the final selected run. Insert ANN regional error table by moneyness-maturity buckets. Insert ANN runtime summary (throughput/latency) using the final benchmark configuration.

6.3. Calibration Network Performance

This section summarizes the calibration workflow results from Chapter 4, focusing on fit quality and operational stability.

The main reported dimensions are:

- objective-function quality at convergence,

- residual statistics versus market quotes,
- robustness diagnostics around the calibrated optimum.

Insert calibration metrics table (objective, weighted MSE, residual RMSE, convergence diagnostics). Insert calibration robustness figure placeholders (e.g., curvature/Jacobian-based diagnostics at $\theta^\star$).

6.4. PINN Pricing Performance

This section reports baseline PINN pricing quality before entering Greek-specific analysis. The goal is to assess whether PDE-constrained training reaches competitive price accuracy and stable residual behavior over the evaluation domain.

6.4.1. Benchmark Setup

The pricing benchmark compares the baseline PINN run against the COS/Heston reference under the same feature conventions and post-processing.

Item	Description
Model run	`outputs/pinn/PINN_mix_scaled_param`
Checkpoint	`train/checkpoints/model_best.pt`
Reference engine	COS/Heston ($N = 1500$, $L = 50$, fallback $L = 80$, `sqrt_t`)
Evaluation sample	10,000 input points, 9,996 valid comparisons
COS invalid/failed cases	14 invalid primary, 4 invalid after fallback
Input scaling	Enabled (`standardize_train`)
Artifacts	`cos_benchmark/metrics.yaml`, `metrics.csv`, `points_pinn_vs_cos.cs`

TABLE 6.1. PINN PRICING BENCHMARK PROTOCOL AND DATASET DEFINITION.

6.4.2. Global Pricing Metrics

Table 6.2 reports the compact global summary used in this chapter.

Metric	Value
n evaluated points	9,996
MSE	3.1621×10^{-5}
RMSE	5.6233×10^{-3}
MAE	2.7458×10^{-3}
Median Abs. Err.	1.0214×10^{-3}
p_{95} Abs. Err.	1.1840×10^{-2}
MAPE (%)	16.0084
Max Abs. Err.	0.1393
Mean error bias	-0.001868
R^2	0.99957

TABLE 6.2. GLOBAL PINN PRICING METRICS VERSUS COS REFERENCE.

6.5. PINN Greeks Performance

This section focuses on sensitivity estimation quality for the baseline PINN and then analyzes the Sobolev extension through a dedicated ablation.

6.5.1. Baseline PINN Greek Accuracy

Baseline Greek quality is evaluated against semi-analytical Heston characteristic-function Greeks, using per-Greek metrics and spatial diagnostics over the benchmark grid.

Insert baseline PINN Greek metrics table (Delta/Gamma/Vega/Theta/Rho). Insert baseline Greek error maps and histograms.

6.5.2. Ablation: Baseline PINN vs Sobolev-Augmented PINN

This subsection reports a controlled ablation focused on Greek accuracy. The objective is to isolate the impact of Jacobian-informed Sobolev terms on top of the baseline PINN loss, comparing:

- **Baseline PINN**: PDE + terminal + lower-boundary losses only.
- **Sobolev-augmented PINN**: baseline loss plus derivative supervision terms $\mathcal{L}_{g,1}$ and $\mathcal{L}_{g,2}$ as defined in Chapter 5.

6.5.3. Experimental Protocol

To make the comparison interpretable, both variants are evaluated under the same benchmark grid and the same reference Greek engine (Heston characteristic-function Greeks).

The ablation is designed so that only the training objective changes, while architecture and evaluation protocol remain fixed.

Confirm and report the exact run identifiers used in this ablation (baseline run dir/checkpoint and Sobolev run dir/checkpoint). Confirm whether both variants use identical sampling seeds and identical collocation subsets. If not, report the exact differences.

6.5.4. Metrics

Greek quality is reported with the same metrics exported by the benchmark pipeline:

$$\text{MSE},\ \text{RMSE},\ R^2,\ \text{MAPE}(\%),\ \text{MAE},\ \max|e|.$$

These metrics are computed per Greek (Δ, Γ, Vega, Θ, Rho). In addition, runtime indicators are reported for training and for batched Greek inference.

Variant	Greek	n	MSE	RMSE	R^2	MAPE (%)	MAE	Max Abs. Err.
Baseline PINN	Delta	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Baseline PINN	Gamma	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Baseline PINN	Vega	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Baseline PINN	Theta	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Baseline PINN	Rho	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Sobolev PINN	Delta	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Sobolev PINN	Gamma	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Sobolev PINN	Vega	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Sobolev PINN	Theta	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Sobolev PINN	Rho	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*

TABLE 6.3. ABLATION METRICS BY GREEK: BASELINE PINN VS SOBOLEV-AUGMENTED PINN (PLACEHOLDER).

6.5.5. Visual Diagnostics

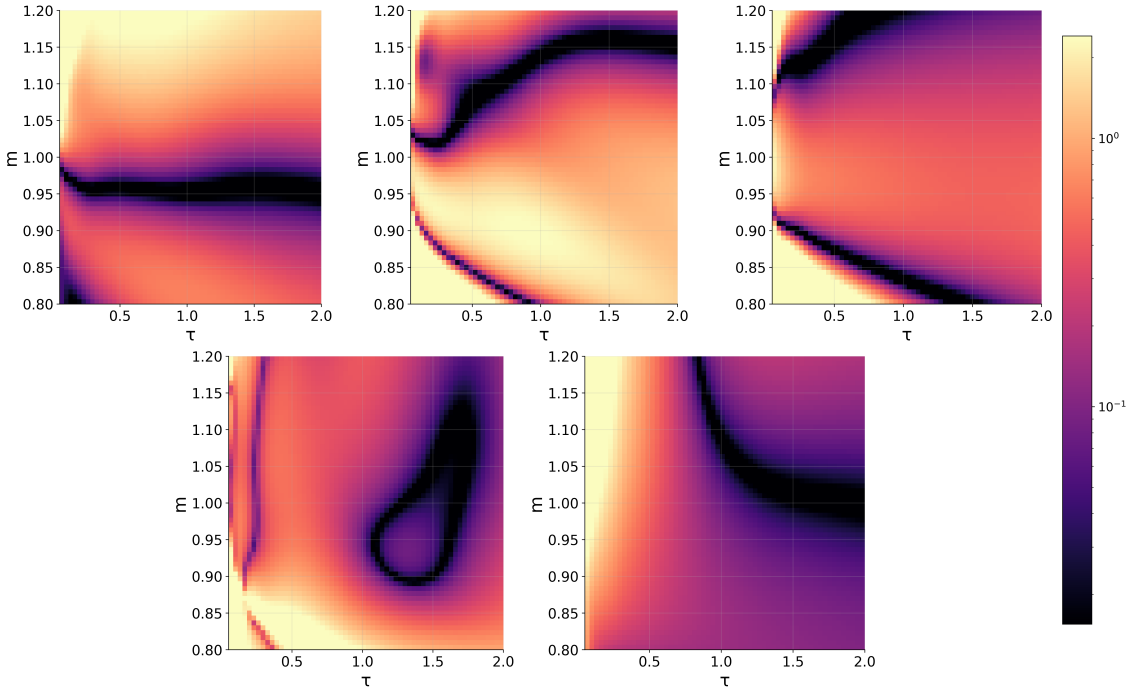


Fig. 6.1. Spatial error for the Greeks computed without the Sobolev method.

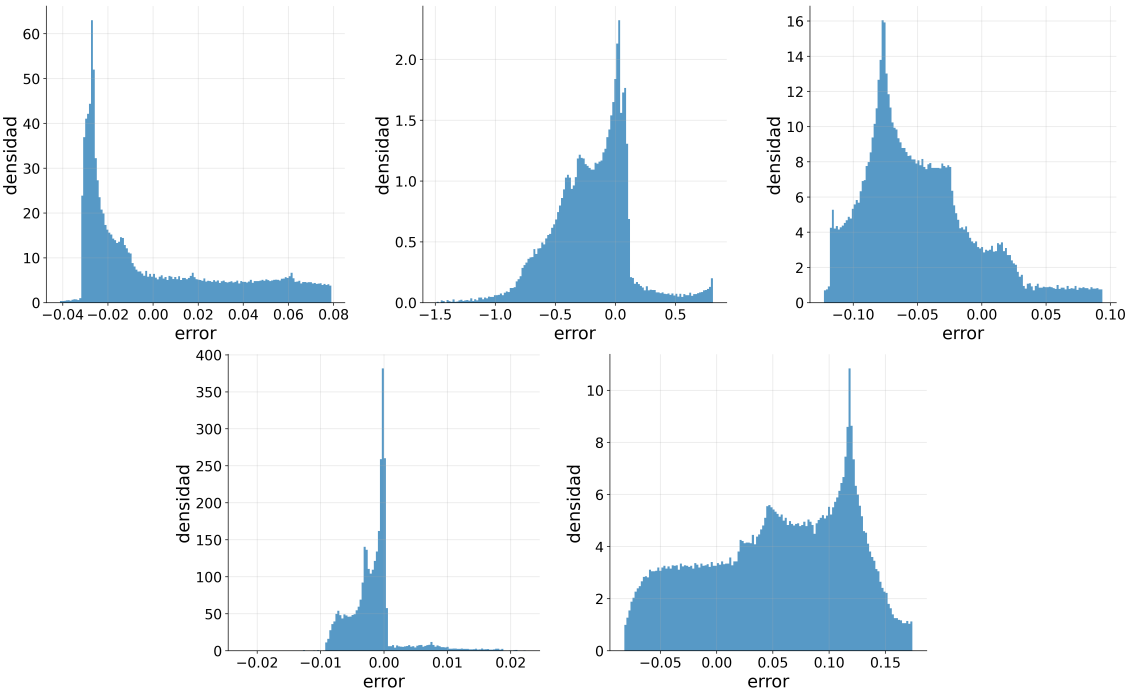


Fig. 6.2. Error-distribution for the Greeks computed witout the Sobolev method.

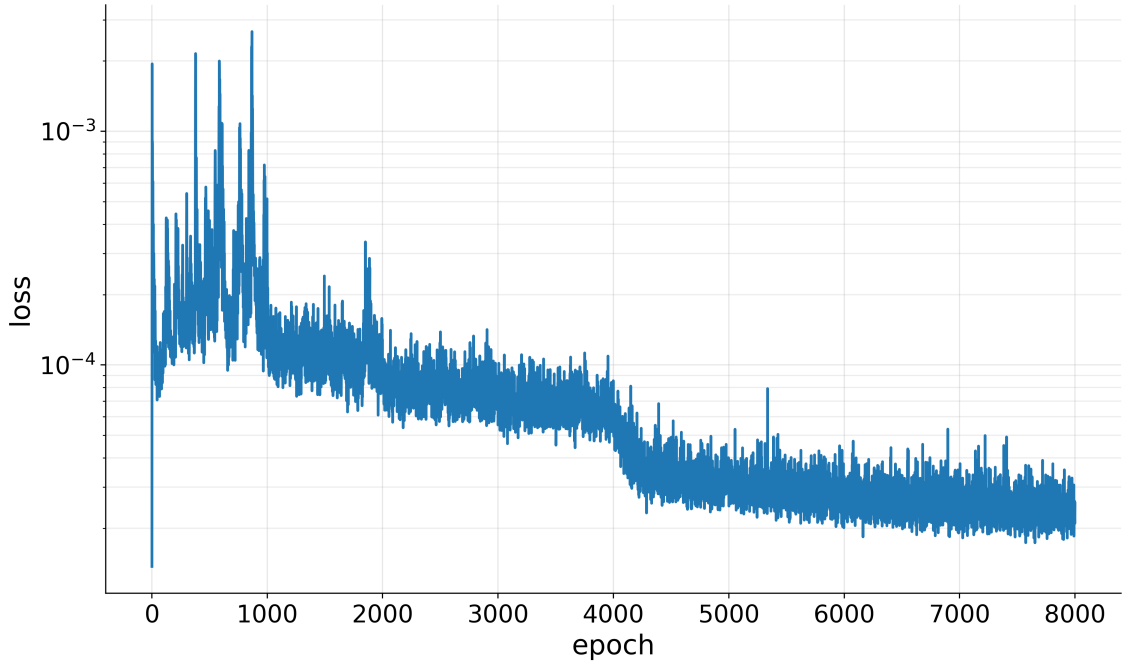


Fig. 6.3. Training dynamics in the ablation study (placeholder).

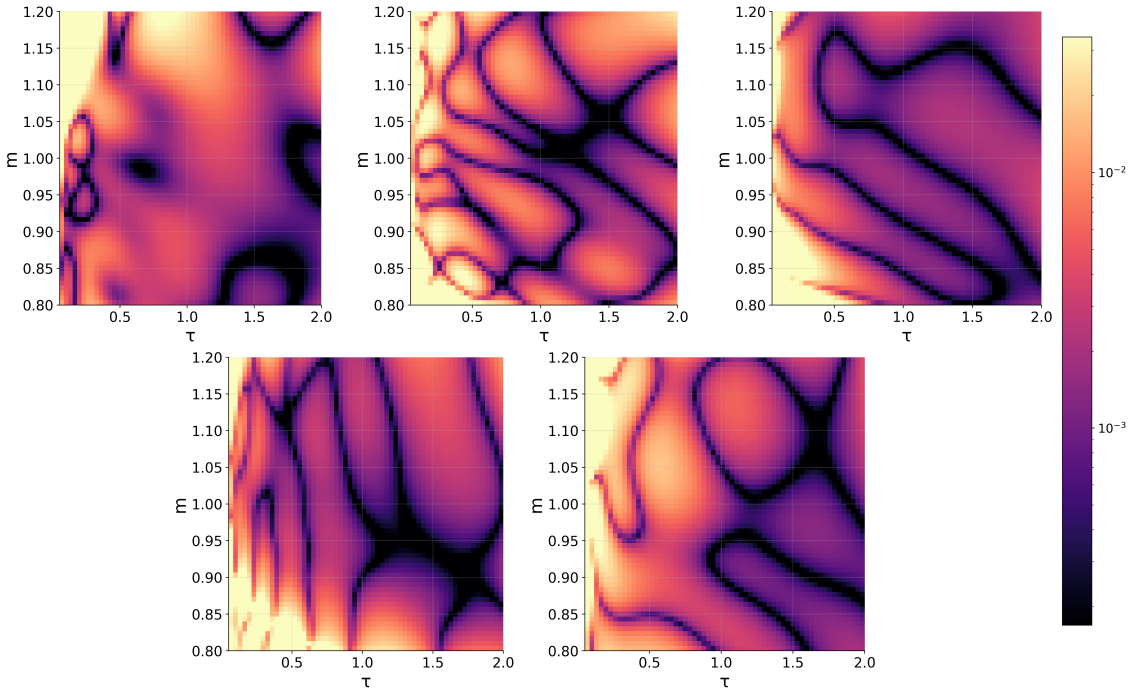


Fig. 6.4. Spatial error for the Greeks computed with the Sobolev method.

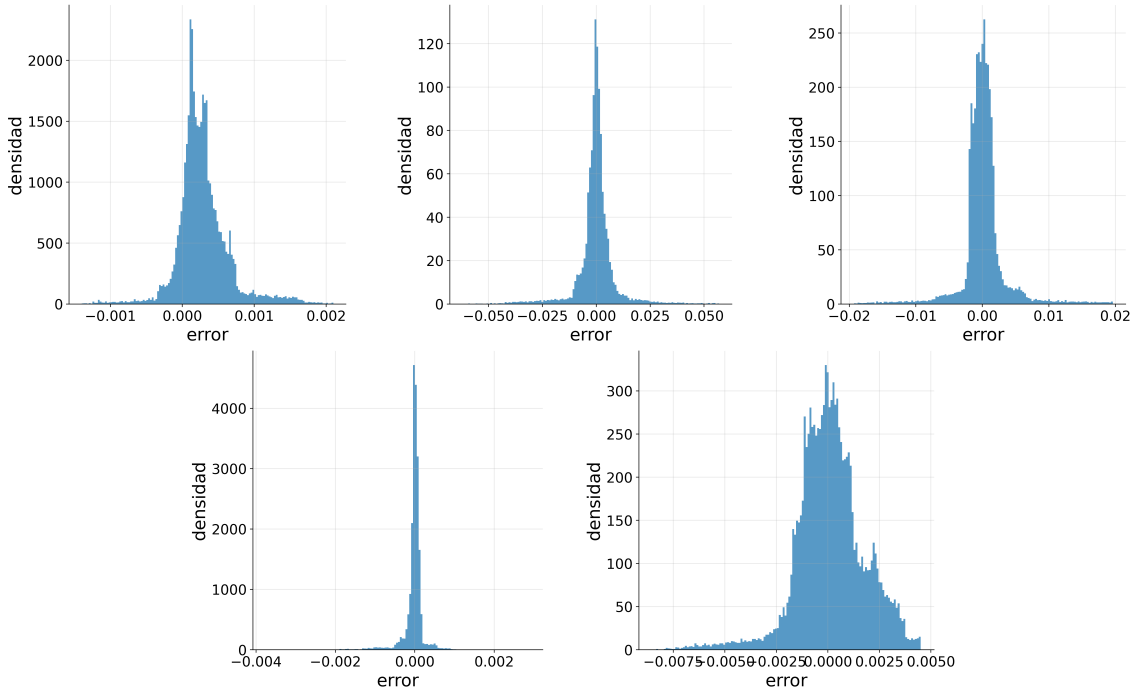


Fig. 6.5. Error-distribution for the Greeks computed with the Sobolev method.

6.6. Cross-Model Computational Comparison

This section provides a horizontal computational comparison across ANN pricing, calibration workflow, and PINN-based pricing/Greeks. The purpose is not to force a single winner, but to clarify where each component is computationally advantageous within the end-to-end research pipeline.

Insert comparison table with representative latency/throughput/cost indicators for ANN, calibration workflow, baseline PINN, and Sobolev PINN. Add one short paragraph on comparability limits (different objectives, output spaces, and optimization stages).

6.7. Generalization And Robustness

This section consolidates robustness evidence across all model blocks, including sensitivity to domain boundaries, extrapolation behavior, and dependence on benchmark design choices.

Insert final robustness summary table across ANN, calibration workflow, and PINN variants.

7. CONCLUSION AND FUTURE WORK

7.1. Market Data

7.2. Exotic Derivatives

7.3. Greeks And Hedging

As a direct continuation of the Greek-focused PINN work, a relevant future direction is to test residual-gradient regularization in the spirit of gPINNs, i.e., adding a penalty on spatial gradients of the PDE residual. This term was not included in the benchmarked implementation of the present thesis and is therefore left for future experiments under controlled ablation settings.

7.4. Model Extension

7.5. Machine Learning Improvements

BIBLIOGRAPHY

[1] B. Oksendal, *Stochastic Differential Equations: An Introduction with Applications*, 6th ed. Springer, 2003.

[2] F. Fang and C. W. Oosterlee, "A novel pricing method for european options based on fourier-cosine series expansions," *SIAM Journal on Scientific Computing*, vol. 31, no. 2, pp. 826–848, 2009. DOI: 10.1137/080718061.

[3] S. L. Heston, "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options," en, *The Review of Financial Studies*, vol. 6, no. 2, pp. 327–343, 1993. DOI: 10.1093/rfs/6.2.327.

[4] F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973. DOI: 10.1086/260062.

[5] F. D. Rouah and S. L. Heston, *The Heston Model and its Extensions in Matlab and C#*. John Wiley & Sons, 2013.

[6] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, 2007.

[7] P. Carr and D. B. Madan, "Option valuation using the fast fourier transform," *Journal of Computational Finance*, vol. 2, no. 4, pp. 61–73, 1999. DOI: 10.21314/JCF.1999.043.

[8] S. Liu, A. Borovykh, L. A. Grzelak, and C. W. Oosterlee, "A neural network-based framework for financial model calibration," en, *Journal of Mathematics in Industry*, vol. 9, no. 1, p. 9, Dec. 2019. DOI: 10.1186/s13362-019-0066-7. Accessed: Oct. 21, 2025. [Online]. Available: https://mathematicsinindustry.springeropen.com/articles/10.1186/s13362-019-0066-7.

[9] D. Hainaut and A. Casas, "Option pricing in the heston model with physics inspired neural networks," en, *Annals of Finance*, vol. 20, no. 3, pp. 353–376, 2024. DOI: 10.1007/s10436-024-00452-7. [Online]. Available: https://doi.org/10.1007/s10436-024-00452-7.

[10] J. Yu, L. Lu, X. Meng, and G. E. Karniadakis, "Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 393, p. 114 823, 2022. DOI: 10.1016/j.cma.2022.114823. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0045782522001438.

[11] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM Journal on Scientific Computing*, vol. 43, no. 5, A3055–A3081, 2021.

[12] S. Wang, X. Yu, and P. Perdikaris, "When and why PINNs fail to train: A neural tangent kernel perspective," *Journal of Computational Physics*, vol. 449, p. 110 768, 2022. DOI: 10.1016/j.jcp.2021.110768. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S002199912100663X.

[13] J. F. Urbán, P. Stefanou, and J. A. Pons, "Unveiling the optimization process of physics informed neural networks: How accurate and competitive can PINNs be?" en, *Journal of Computational Physics*, vol. 523, p. 113 656, Feb. 2025. DOI: 10.1016/j.jcp.2024.113656. Accessed: Mar. 19, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0021999124009045.

[14] M. D. Mckay, R. J. Beckman, and W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output From a Computer Code," en, *Technometrics*, vol. 42, no. 1, pp. 55–61, Feb. 2000. DOI: 10.1080/00401706.2000.10485979. Accessed: Mar. 21, 2026. [Online]. Available: http://www.tandfonline.com/doi/abs/10.1080/00401706.2000.10485979.

[15] R. B. Gramacy, *Surrogates: Gaussian Process Modeling, Design, and Optimization for the Applied Sciences*. Chapman and Hall/CRC, 2020.

[16] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989. DOI: 10.1007/BF02551274.

[17] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991. DOI: 10.1016/0893-6080(91)90009-T.

[18] W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, and R. Pascanu, "Sobolev Training for Neural Networks," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. [Online]. Available: https://papers.nips.cc/paper/7015-sobolev-training-for-neural-networks.

A. ANALYSIS OF THE COS METHOD ERROR

no se si dejarlo como apéndice o meterlo en alguna seccion

For the COS method, we have three sources of error:

- The integration range truncation error, ε_1.
- The series truncation error on $[a, b]$, ε_2.
- The error related to approximating $\bar{A}_k(x)$ by $\bar{F}_k(x)$, ε_3 (poner ref a alguna ecuacion).

It is important to note that there is no error in the payoff coefficient H_k for plain vanilla European options.

If the integration range $[a, b]$ is chosen sufficiently large, the overall error is dominated by ε_2 (TODO: argumentarlo). Also, ε_2 converges exponentially if $f_X(x) \in C^\infty[a, b]$ (TODO: justificarlo tamb). Otherwise, converges only algebraically.

To choose an optimal integration range we will use the expression (TODO: ref a Oorsterlee)

$$[a, b] = \left[(x + \zeta_1) - L\sqrt{\zeta_2 + \sqrt{\zeta_4}},\ (x + \zeta_1) + L\sqrt{\zeta_2 + \sqrt{\zeta_4}} \right], \tag{A.1}$$

where $L \in [6, 12]$ is a parameter that depends on the tolerance we want to have, and ζ_i are the cumulants, computed using the expression (2.8).

B. CALIBRATION OF THE HESTON MODEL

TODO:

C. GLOBAL NOTATION GLOSSARY

This appendix contains a global glossary of symbols used throughout the thesis. It is intended as a living reference and can be expanded as additional notation appears in later revisions.

TABLE C.1. GLOBAL NOTATION GLOSSARY (V1)

Symbol	Meaning
$t,\ T,\ \tau$	Valuation time, maturity, and time to maturity ($\tau = T - t$).
$S_t,\ K,\ m$	Spot price, strike, and moneyness ($m = S_0/K$).
r	Constant risk-free interest rate.
$\mathbb{Q}$	Risk-neutral probability measure.
V_t	Arbitrage-free option value at time t.
Ψ	Option payoff function at maturity.
X_t	State vector of model factors (e.g., log-price and variance).
θ	Generic model-parameter vector.
$\kappa,\ \bar{v},\ \gamma,\ \rho,\ v_0$	Heston parameters: mean reversion, long-run variance, vol-of-vol, correlation, and initial variance.
$\phi_X(u)$	Characteristic function of random variable X.
ζ_k	k-th cumulant of the log-price distribution.
σ_{imp}	Implied volatility obtained from numerical inversion.

Source: Author's elaboration